App Dev II Project Report

1. Student Details

Name: Gyan Prakash Kushwaha
Roll Number: 23f3004091
Email: 23f3004091@ds.study.iitm.ac.in
About Me: I am a student at the IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies.

---

2. Project Details

Project Title: Trekking Management System (TMS v2)
Problem Statement: To design and develop a Trekking Management System (TMS) that streamlines trekking operations through a secure, role-based web application. The system supports three core user roles—Admin, Trek Staff, and Trekker—and enables each role to perform domain-specific tasks such as user and profile management, trek scheduling, booking handling, and trekker history tracking.
Approach: The Trekking Management System is a full-stack web application built with a modular architecture that separates backend APIs, data models, and frontend UI layers for maintainability and scalability.

- **Backend Framework:** The backend is developed using Flask (Python), with a clean route, model, and task separation. It handles authentication, authorization, scheduling, business rules, and REST API endpoints.
- **Data Management:** SQLAlchemy is used as the ORM with an SQLite database. The schema models key entities including User, Trekker, StaffProfile, Trek, and Booking, along with their relationships and lifecycle states (e.g., Pending, Approved, Completed, Cancelled).
- **Authentication and Security:** JWT-based authentication is implemented using Flask-JWT-Extended, with password security handled via Werkzeug utilities. Protected routes enforce authenticated access and role-level permissions.

Role-Based Access Control (RBAC): The system is centered on three roles:

- **Admin:** Manages treks, oversees staff and trekkers (including approvals and blacklisting), and monitors overall operational data and bookings.
- **Trek Staff:** Manages their own staff profile, views and updates assigned treks, and tracks bookings associated with their assigned treks.
- **Trekker:** Browses available treks, manages their profile, books treks, and views their booking and trek history.

Task Scheduling and Notifications: Celery is integrated for asynchronous and scheduled tasks, with Redis as the broker and result backend. Automated jobs include periodic reporting and task offloading (e.g., CSV export generations). Flask-Mail is configured for email delivery workflows.
Caching Layer: Redis is also used for API response caching to improve read performance on frequently accessed endpoints.
Frontend Development: The frontend is built using Vue 3 with Vue Router and Axios, powered by Vite for development and build workflows. State management is handled by Pinia. UI styling utilizes Bootstrap 5 to provide a responsive, user-friendly interface across admin, staff, and trekker dashboards.
Overall Outcome: TMS v2 provides an end-to-end digital workflow for trekking administration, staff coordination, and trekker self-service, with an extensible architecture suitable for future enhancements such as advanced analytics, audit logging, and production-grade email providers.

---

3. AI/LLM Declaration

I used Copilot to assist in writing CSS for the frontend of my website, creating dummy data for testing different features of my application, and improving variable naming consistency. The extent of AI/LLM usage is around 5–10%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework used to build REST APIs, handle routing, and implement business logic.
Flask-SQLAlchemy	Object Relational Mapper (ORM) used to interact with the SQLite database through Python models.
SQLite	Relational database for local data storage.
Bootstrap 5	Frontend CSS framework used for responsive layout, reusable UI components, and consistent styling.
Flask-JWT-Extended	JWT-based authentication and role-protected API access.
Werkzeug Security	Password hashing and verification.
Flask-CORS	Enables secure cross-origin requests between frontend and backend.

Celery	Background job processing and scheduled tasks.
Redis	Celery broker/result backend and caching layer.
Flask-Mail	Email integration layer.
Vue 3	Frontend framework for SPA UI.
Pinia	State management library for Vue 3.
Vue Router	Client-side routing/navigation.
Axios	API calls from frontend to Flask backend.
Vite	Frontend dev server and fast build tooling.

---

5. Database Schema / ER Diagram

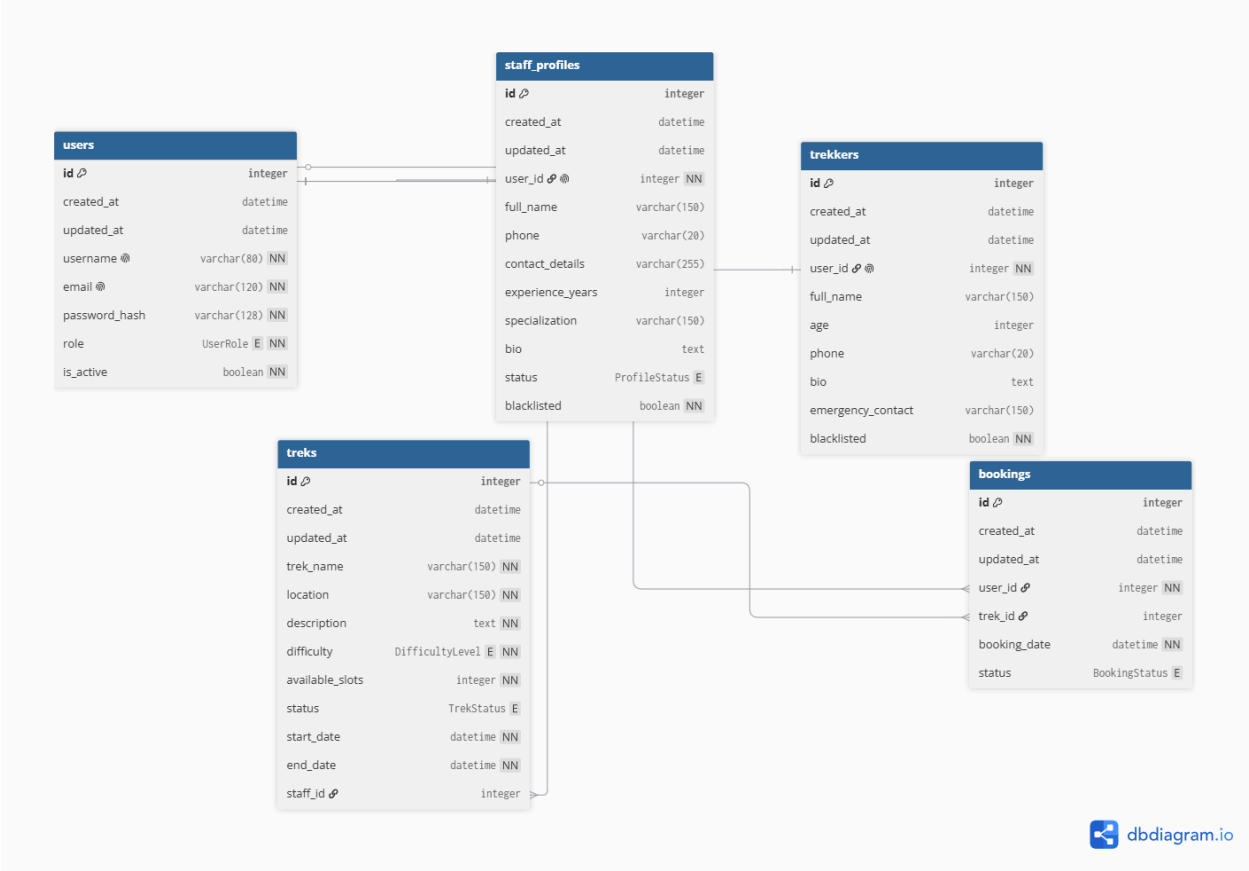
Tables

- **User:** Stores authentication and profile details for all system users (Admin, Trek Staff, Trekker). id (PK), username (UNIQUE), email (UNIQUE), password_hash, role (Admin / Trek Staff / Trekker), is_active, created_at, updated_at
- **Trekker:** Stores trekker-specific profile information. id (PK), user_id (FK → users.id), full_name, age, phone, bio, emergency_contact, blacklisted, created_at, updated_at
- **StaffProfile:** Stores trek staff-specific information. id (PK), user_id (FK → users.id), full_name, phone, contact_details, experience_years, specialization, bio, status (Pending / Approved / Rejected), blacklisted, created_at, updated_at
- **Trek:** Stores details of the treks available. id (PK), trek_name, location, description, difficulty (Easy / Moderate / Hard), available_slots, status (Pending / Approved / Open / Ongoing / Closed / Cancelled / Completed), start_date, end_date, staff_id (FK → users.id), created_at, updated_at
- **Booking:** Stores booking information mapping trekkers to treks. id (PK), user_id (FK → users.id), trek_id (FK → treks.id), booking_date, status (Booked / Cancelled / Completed), created_at, updated_at

Relationships

- **One-to-One → User → Trekker:** Each Trekker profile belongs to exactly one User account.
- **One-to-One → User → StaffProfile:** Each Staff Profile belongs to exactly one User account.
- **One-to-Many → User (Trek Staff) → Trek:** One Staff member can manage many Treks. Each Trek is assigned to one Staff member.
- **One-to-Many → User (Trekker) → Booking:** One Trekker can make multiple Bookings.

- **One-to-Many → Trek → Booking:** One Trek can have multiple Bookings associated with it.



---

6. API Resource Endpoints

Authentication Routes (/api/auth)

- POST /register-trekker - Register a new Trekker
- POST /login - User login
- POST /refresh - Refresh JWT token

Admin Routes (/api/admin)

- GET /summary - Admin dashboard statistics
- GET /staff - List all staff profiles
- POST /register-staff - Register a new Staff member
- PATCH /approve-staff - Approve or reject staff profile
- PATCH /blacklist-staff - Blacklist a staff member
- GET /trekkers - List all trekkers
- PATCH /blacklist-trekker - Blacklist a trekker
- GET /treks - List all treks
- POST /create-trek - Create a new trek
- PUT /update-trek/:id - Fully update a trek

- PATCH /manage-trek/:id - Partially update trek status
- DELETE /delete-trek/:id - Delete a trek
- GET /bookings - List all bookings
- PATCH /manage-booking/:id - Update booking status

Trek Staff Routes (/api/staff)

- GET /summary - Staff dashboard summary
- GET /profile - Get staff profile
- PATCH /profile - Update staff profile
- GET /treks - List assigned treks
- PATCH /update-trek/:id - Update assigned trek status
- GET /bookings/:trek_id - Get bookings for a specific trek
- PATCH /update-booking-status/:id - Update booking status
- GET /user/:trek_id - Get user details for a trek

Trekker Routes (/api/trekker)

- GET /summary - Trekker dashboard summary
- GET /profile - Get trekker profile
- PATCH /profile - Update trekker profile
- GET /treks - Browse available treks
- POST /book-trek/:trek_id - Book a trek
- GET /history - Get booking/trek history
- PATCH /update-booking-status/:id - Cancel or update a booking
- POST /export-history - Trigger background CSV export of history
- GET /export-status/:task_id - Check status of CSV export task
- GET /download-csv/:task_id - Download the generated CSV

---

7. Video Presentation

Drive Link:
https://drive.google.com/drive/folders/1aaKyvGcohn0JHrLYpyYu4Vwe_jnDDEjz?usp=sharing